

Algorithms for Data Science

Exam 2026

Group B
(Solutions)

June 2026

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	10	15	15	15	20	15	20	15	15	10	15	15	10	180
Score:														

*The exam has 13 tasks worth **180 points** in total. Duration: 2 hours. No aids allowed.*

1. **Asymptotic notation.** Simplify each expression to the simplest possible Θ form.

[10]

(a) $\Theta(4n^2 + 200n + \log^2 n)$

[2]

Answer: _____

Solution: $\Theta(n^2)$. The term $4n^2$ dominates both $200n$ and $\log^2 n$ for large n .

(b) $\Theta(\sqrt{n} \cdot \log n + \sqrt{n})$

[2]

Answer: _____

Solution: $\Theta(\sqrt{n} \log n)$. Factor out $\sqrt{n}$: $\sqrt{n}(\log n + 1)$. Since $\log n + 1 = \Theta(\log n)$, the result is $\Theta(\sqrt{n} \log n)$.

(c) $\Theta(n! + 3^n)$

[3]

Answer: _____

Solution: $\Theta(n!)$. Factorials grow faster than any exponential: $n! \geq 3^n$ for all sufficiently large n (since $n! = n \cdot (n-1) \cdots 1$ and each factor eventually exceeds 3).

(d) Give the time complexity of the following code in Θ notation:

[3]

```

1 for i = 1 to n do
2   j ← 1
3   while j ≤ n do
4     print(i + j)
5     j ← j · 2

```

Answer: _____

Solution: $\Theta(n \log n)$. The outer loop runs n times. The inner while loop doubles j each iteration, so it runs $\lfloor \log_2 n \rfloor + 1 = \Theta(\log n)$ times. Total: $\Theta(n \log n)$.

2. Recurrences.

[15]

- (a) Consider the following pseudocode for ALGOA:

[5]

```

1 function ALGOA( $n$ )
2   if  $n \leq 1$  then
3     return
4    $i \leftarrow n$ 
5   while  $i > n - 4$  do
6     ALGOA( $\lfloor n/15 \rfloor$ )
7      $i \leftarrow i - 1$ 
8   for  $j = 1$  to 12 do
9     ALGOA( $\lfloor n/15 \rfloor$ )
10  for  $k = 1$  to  $n^8$  do
11    doWork()

```

Write the recurrence $T(n)$ for the worst-case running time of ALGOA(n). Clearly count the number of recursive calls made.

Solution: The **while** loop runs while $i > n - 4$, starting at $i = n$. It executes for $i = n, n - 1, n - 2, n - 3$ (stops when $i = n - 4$), making exactly **4** recursive calls. The **for** loop makes **12** more recursive calls. Total: $4 + 12 = 16$ calls, each to ALGOA($\lfloor n/15 \rfloor$). The last for loop contributes $\Theta(n^8)$ non-recursive work.

$$T(n) = 16T\left(\left\lfloor \frac{n}{15} \right\rfloor\right) + \Theta(n^8).$$

- (b) Solve the following recurrence using the
- Master Theorem**
- . Show which case applies and explain.

[5]

$$T(n) = 27T\left(\frac{n}{3}\right) + \Theta(n^2)$$

Solution: $a = 27, b = 3, f(n) = \Theta(n^2)$.

$c = \log_b a = \log_3 27 = 3$.

Since $f(n) = \Theta(n^2)$ and $c = 3 > 2$, Case A of the Master Theorem applies.

Result: $T(n) = \Theta(n^c) = \Theta(n^3)$.

- (c) Solve the following recurrence using the
- Master Theorem**
- . Show which case applies and explain.

[5]

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

Solution: $a = 2, b = 2, f(n) = \Theta(n)$.

$c = \log_2 2 = 1$.

Since $f(n) = \Theta(n) = \Theta(n^c)$, Case B applies.

Result: $T(n) = \Theta(n \log n)$.

3. Sorting algorithms.

[15]

- (a) We have an array of $n = 10^6$ integers, where each integer is in the range $[0, 999]$. Which sorting algorithm would you use to sort this array in the most efficient way? Justify your choice and state its time complexity. [5]

Solution: Counting Sort (or Radix Sort). Since all values are in $[0, 999]$, we can use Counting Sort with $k = 1000$ buckets. Time complexity: $\Theta(n + k) = \Theta(n)$ (since $k = 1000$ is a constant relative to $n = 10^6$). This beats comparison-based sorting which requires $\Omega(n \log n)$.

- (b) Consider the following array: $A = \langle 3, 1, 4, 1, 5, 9, 2, 6 \rangle$. [5]

Show the state of the array after **each partition step** of QuickSort, using the **last element** as the pivot. Show only the first two levels of recursion (i.e., partition the full array once, then partition each resulting sub-array once).

Solution: Level 0: Full array $\langle 3, 1, 4, 1, 5, 9, 2, 6 \rangle$. Pivot = 6.
 After partition: $\langle 3, 1, 4, 1, 5, 2, \mathbf{6}, 9 \rangle$ (elements ≤ 6 on left, pivot at position 6, elements > 6 on right).
Level 1a: Left sub-array $\langle 3, 1, 4, 1, 5, 2 \rangle$. Pivot = 2.
 After partition: $\langle 1, 1, \mathbf{2}, 3, 4, 5 \rangle$. (Elements ≤ 2 left, pivot at pos 2, rest right.)
Level 1b: Right sub-array $\langle 9 \rangle$ already sorted (single element).

- (c) Is it possible to sort n elements using only $O(n \log \log n)$ comparisons in the *worst* case? Explain your answer. [5]

Solution: No. The comparison-based sorting lower bound states that any comparison-based algorithm requires $\Omega(n \log n)$ comparisons in the worst case. Since $n \log \log n = o(n \log n)$, no comparison-based algorithm can achieve $O(n \log \log n)$ comparisons in the worst case.

4. Dynamic programming: stair climbing.

[15]

You are climbing a staircase with n steps. Each move you may climb either **1** or **3** steps. Let $dp[n]$ be the number of distinct ways to reach step n (starting at step 0).

- (a) The recurrence is $dp[n] = dp[n-1] + dp[n-3]$ for $n \geq 3$. State the base cases $dp[0]$, $dp[1]$, and $dp[2]$, and briefly explain why the recurrence holds.

[3]

Solution: $dp[0] = 1$ (one way to stand at the start without moving), $dp[1] = 1$ (only a single 1-step), $dp[2] = 1$ (only two 1-steps, since a 3-step is impossible).
 The recurrence holds because the last move to reach step n was either a 1-step (contributing $dp[n-1]$ ways) or a 3-step (contributing $dp[n-3]$ ways); these cases are disjoint and exhaustive.
 For $n < 3$ a 3-step is impossible, so only the single-step path exists.

- (b) Using the base cases and the recurrence, fill in the DP table for $n = 0$ to 7.

[7]

n	0	1	2	3	4	5	6	7
$dp[n]$								

Solution: $dp[0] = 1$, $dp[1] = 1$, $dp[2] = 1$, $dp[3] = 2$, $dp[4] = 3$, $dp[5] = 4$, $dp[6] = 6$, $dp[7] = 9$.

- (c) How many distinct ways are there to climb 7 steps?

[3]

Answer: _____

Solution: $dp[7] = 9$.

- (d) What is the time complexity of computing $dp[0 \dots n]$ using the recurrence?

[2]

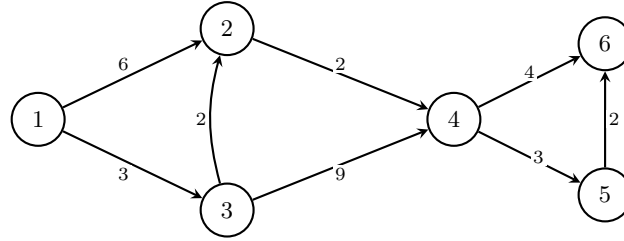
Answer: _____

Solution: $\Theta(n)$. Each entry is computed exactly once in a single left-to-right pass using the previous two values.

5. Graph algorithms.

[20]

Consider the weighted directed graph below with vertices $\{1, 2, 3, 4, 5, 6\}$ and edges: $1 \rightarrow 2$ (6), $1 \rightarrow 3$ (3), $2 \rightarrow 4$ (2), $3 \rightarrow 2$ (2), $3 \rightarrow 4$ (9), $4 \rightarrow 5$ (3), $4 \rightarrow 6$ (4), $5 \rightarrow 6$ (2).



(a) Run **Dijkstra's algorithm** starting from vertex 1. Fill in the table below.

[8]

Step	Extracted	$d[1]$	$d[2]$	$d[3]$	$d[4]$	$d[5]$	$d[6]$
0 (init)	—	0	∞	∞	∞	∞	∞
1							
2							
3							
4							
5							
6							

Solution: **Step 1:** Extract 1 ($d = 0$). Update: $d[2] = 6$, $d[3] = 3$.
Step 2: Extract 3 ($d = 3$). Update: $d[2] = \min(6, 5) = 5$, $d[4] = 12$.
Step 3: Extract 2 ($d = 5$). Update: $d[4] = \min(12, 7) = 7$.
Step 4: Extract 4 ($d = 7$). Update: $d[5] = 10$, $d[6] = 11$.
Step 5: Extract 5 ($d = 10$). Update: $d[6] = \min(11, 12) = 11$ (no change).
Step 6: Extract 6 ($d = 11$). No updates.
 Final: $d[1] = 0$, $d[2] = 5$, $d[3] = 3$, $d[4] = 7$, $d[5] = 10$, $d[6] = 11$.

(b) What is the shortest path from vertex 1 to vertex 6? List the vertices and total distance.

[5]

Solution: Path: $1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 6$, total distance = $3 + 2 + 2 + 4 = 11$.

(c) What is the time complexity of Dijkstra's algorithm with a **binary min-heap**? Express in terms of n (vertices) and m (edges).

[4]

Solution: $\Theta((n + m) \log n)$.

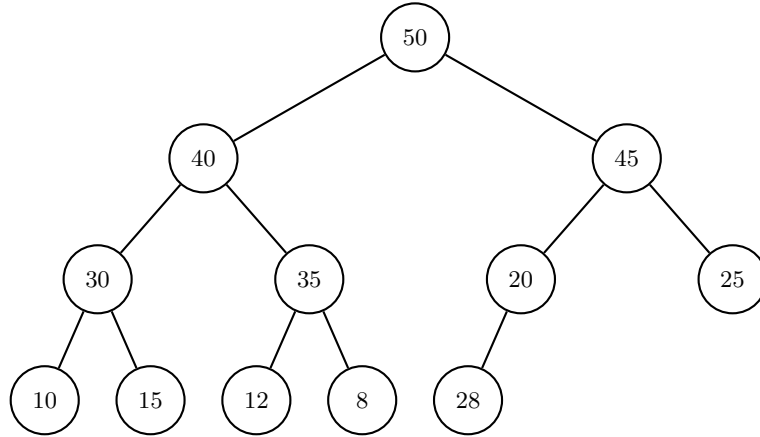
(d) Run **BFS** (ignoring edge weights) from vertex 1. What hop-count distances does BFS report? Give one example where BFS and Dijkstra disagree and explain why.

[3]

Solution: BFS: $d[1] = 0$, $d[2] = 1$, $d[3] = 1$, $d[4] = 2$, $d[5] = 3$, $d[6] = 3$.
 Example: BFS gives $d[2] = 1$ (direct edge), but Dijkstra gives $d[2] = 5$ via $1 \rightarrow 3 \rightarrow 2$ (weight $3 + 2 = 5 < 6$). BFS ignores weights and finds fewest-hop paths only.

6. Heaps and Binary Search Trees.

- (a) The array $A = [50, 40, 45, 30, 35, 20, 25, 10, 15, 12, 8, 28]$ (indexed 1..12) is claimed to represent a **max-heap**. The corresponding complete binary tree is drawn below.

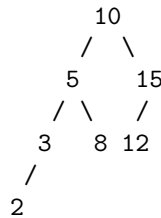


- (i) Identify the *one* index where the max-heap property is **violated** (a node is greater than its parent). (2 pts)
 (ii) Fix the violation by changing *exactly one* value. State which index and to what value. (3 pts)

Solution: (i) **Violation:** $A[12] = 28 > A[6] = 20$. Index **12**.
 (ii) **Fix:** Change $A[12]$ to any value ≤ 20 , e.g. set $A[12] = 18$.

- (b) Starting from the root key **10**, insert the keys $\langle 5, 15, 3, 8, 12, 2 \rangle$ (in this order) into a **Binary Search Tree**. Draw the resulting BST.

Solution:



Insertions: 5 (left of 10), 15 (right of 10), 3 (left of 5), 8 (right of 5), 12 (left of 15), 2 (left of 3).

- (c) Check whether the BST from part (b) is also an **AVL tree**. Compute the height of each subtree (height of a leaf = 0, height of empty subtree = -1) and verify the balance condition $|h_L - h_R| \leq 1$ at every node.

Solution: Heights: $h(2) = 0$, $h(3) = 1$, $h(8) = 0$, $h(5) = 2$, $h(12) = 0$, $h(15) = 1$, $h(10) = 3$.
 Balance factors:

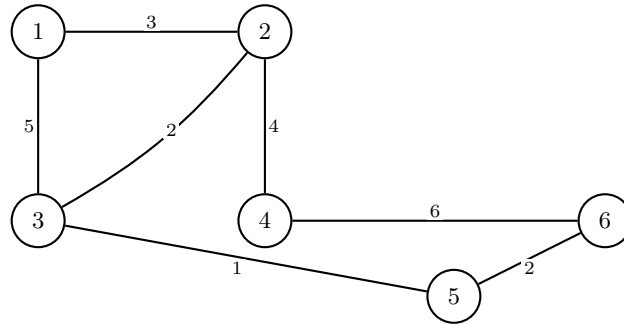
- Node 10: $|h(5) - h(15)| = |2 - 1| = 1$
- Node 5: $|h(3) - h(8)| = |1 - 0| = 1$
- Node 3: $|h(2) - (-1)| = |0 - (-1)| = 1$
- Node 15: $|h(12) - (-1)| = |0 - (-1)| = 1$

All balance factors ≤ 1 . The BST **is** an AVL tree (though not perfectly symmetric — every node has balance factor exactly 1 or 0).

7. Minimum spanning tree — Kruskal's algorithm.

[20]

Consider the following weighted *undirected* graph with vertices $\{1, 2, 3, 4, 5, 6\}$:



- (a) List all 7 edges sorted by weight in **non-decreasing** order.

[3]

Solution: 3-5 (1), 2-3 (2), 5-6 (2), 1-2 (3), 2-4 (4), 1-3 (5), 4-6 (6).

- (b) Run **Kruskal's algorithm**. For each edge (in sorted order), state whether it is **added** or **rejected**, and write the resulting connected components.

[10]

Step	Edge	Add / Reject	Components after this step
1			
2			
3			
4			
5			
6			
7			

Solution:

Step	Edge	Action	Components
1	3-5 (1)	Add	$\{1\}, \{2\}, \{3, 5\}, \{4\}, \{6\}$
2	2-3 (2)	Add	$\{1\}, \{2, 3, 5\}, \{4\}, \{6\}$
3	5-6 (2)	Add	$\{1\}, \{2, 3, 5, 6\}, \{4\}$
4	1-2 (3)	Add	$\{1, 2, 3, 5, 6\}, \{4\}$
5	2-4 (4)	Add	$\{1, 2, 3, 4, 5, 6\}$ (MST complete)
6	1-3 (5)	Reject	1 and 3 already connected
7	4-6 (6)	Reject	4 and 6 already connected

- (c) Draw the resulting MST.

[3]

Solution: MST edges: 3-5 (1), 2-3 (2), 5-6 (2), 1-2 (3), 2-4 (4). Total weight = $1+2+2+3+4 = 12$.

- (d) What is the total weight of the MST?

[2]

Answer: _____

Solution: 12.

- (e) What is the time complexity of Kruskal's algorithm in terms of n and m ?

[2]

Solution: $\Theta(m \log m)$, dominated by sorting the edges.

8. Complexity theory: **P** and **NP**.

[15]

- (a) Briefly define the complexity classes **P** and **NP**. What does it mean for a problem to be **NP-complete**? [5]

Solution: P: The class of decision problems solvable by a deterministic algorithm in polynomial time.

NP: The class of decision problems for which a proposed solution (certificate) can be *verified* in polynomial time by a deterministic algorithm.

NP-complete: A problem X is NP-complete if (1) $X \in \text{NP}$, and (2) every problem in NP can be reduced to X in polynomial time (X is NP-hard). It is among the hardest problems in NP.

- (b) For each problem below, state whether it is **in P** (an efficient polynomial-time algorithm is known) or **NP-complete** (no polynomial-time algorithm is known). Give a one-sentence justification. [6]

- (i) Binary search in a sorted array of n elements.
- (ii) Checking whether a graph is bipartite (2-colorable).
- (iii) Independent set decision: given a graph G and integer k , does G have an independent set of size $\geq k$?
- (iv) Knapsack decision: given items with weights and values, a capacity W , and a value threshold V , does a subset of items fit within capacity and achieve value $\geq V$?
- (v) Finding a minimum spanning tree of a weighted graph.

Solution: (i) **P** — binary search runs in $O(\log n)$.

(ii) **P** — BFS/DFS 2-coloring runs in $O(n + m)$.

(iii) **NP-complete** — no polynomial-time algorithm is known; the best exact algorithms are exponential.

(iv) **NP-complete** — the knapsack decision problem is NP-complete; it captures the hardness of subset selection.

(v) **P** — Kruskal's or Prim's algorithm runs in $O(m \log m)$.

- (c) Mark **all** correct statements. [4]

- ☒ **If problem X is NP-complete and $X \in \text{P}$, then $\text{P} = \text{NP}$.**
- ☐ NP-complete problems cannot be solved exactly, only approximated.
- ☒ **$\text{P} \subseteq \text{NP}$.**
- ☒ **Every NP-complete problem can be reduced to any other NP-complete problem in polynomial time.**
- ☐ Hamiltonian cycle is in P.
- ☒ **SAT was the first problem proven NP-complete.**

Solution: Correct: 1st, 3rd, 4th, 6th. (2nd is false: NP-complete problems can often be solved exactly, just not in polynomial time in general; 5th is false: Hamiltonian cycle is NP-complete.)

9. Choosing the right data structure.

[15]

For each of the three scenarios below, choose the **most appropriate** data structure from the list and give a brief explanation. In your answer, state the relevant time and/or space complexity.

Available data structures:

1. Unsorted array
2. Sorted array
3. Hash table
4. Max-heap / Priority queue
5. BST (unbalanced)
6. AVL tree
7. Find-Union (Disjoint Set Union)
8. Queue

- (a) A compiler processes a source file containing up to n variable declarations. During compilation, it must **insert** each variable name into a symbol table and **look up** any variable by name, both in $O(1)$ average time. The names have no natural ordering, and sorted output or range queries are never required.

[5]

Most appropriate data structure: _____

Solution: Hash table. Supports average-case $O(1)$ insert and lookup by key. No ordering is needed, so a BST or AVL tree (which give $O(\log n)$) would be slower on average. A sorted array would require $O(\log n)$ search and $O(n)$ insertion. Space: $\Theta(n)$.

- (b) A web server handles incoming HTTP requests one at a time. Requests must be processed in **exactly the order they arrive** (first-in, first-out). The only two operations required are: enqueue a new request when it arrives, and dequeue the next request to process.

[5]

Most appropriate data structure: _____

Solution: Queue. Directly implements FIFO semantics with $O(1)$ enqueue and dequeue (using a circular array or linked-list implementation). A heap or AVL tree would be overkill — there is no priority ordering and no search by key. Space: $\Theta(n)$.

- (c) An airline's booking system maintains n reservations keyed by booking ID. The system must support **insert**, **delete**, and **exact-match search by ID**, each in $O(\log n)$ *worst-case* time. Additionally, it must **print all reservations sorted by ID** in $O(n)$ time on demand.

[5]

Most appropriate data structure: _____

Solution: AVL tree. Guarantees $O(\log n)$ worst-case insert, delete, and search via self-balancing rotations. An unbalanced BST degrades to $O(n)$ in the worst case. In-order traversal yields sorted output in $O(n)$. A hash table gives $O(1)$ average search but cannot produce sorted output. Space: $\Theta(n)$.

10. Hash tables.

[10]

- (a) Insert the keys
- $\langle 6, 13, 2, 9, 20, 4 \rangle$
- (in this order) into a hash table of size 7 using the hash function

[4]

$$h(k) = k \bmod 7.$$

Collisions are resolved by **separate chaining** (each slot stores a linked list). Draw the resulting hash table, showing all chains.

Solution: $h(6) = 6$, $h(13) = 6$, $h(2) = 2$, $h(9) = 2$, $h(20) = 6$, $h(4) = 4$.

Slot	Chain
0	(empty)
1	(empty)
2	$2 \rightarrow 9$
3	(empty)
4	4
5	(empty)
6	$6 \rightarrow 13 \rightarrow 20$

- (b) What is the
- worst-case**
- time complexity for a search operation in a hash table with chaining? When does this worst case occur?

[3]

Solution: Worst case: $\Theta(n)$, when all n keys hash to the same slot (one chain of length n). This occurs, for example, with adversarial input tailored to the hash function, or if all keys are congruent modulo the table size.

- (c) Briefly compare a
- hash table**
- and an
- AVL tree**
- for the following operations. State the complexity and name one advantage of each data structure over the other.

[3]

Operation	Hash table	AVL tree
Search		
Insert		
Delete		
Sorted output of all keys		

Solution:	Operation	Hash table	AVL tree
	Search	$O(1)$ expected	$O(\log n)$
	Insert	$O(1)$ amortized	$O(\log n)$
	Delete	$O(1)$ expected	$O(\log n)$
	Sorted output	$O(n \log n)$ (sort first)	$O(n)$ (in-order)

Hash table advantage: $O(1)$ average operations — faster for simple insert/search/delete.

AVL tree advantage: supports sorted output in $O(n)$ and all operations in $O(\log n)$ *worst case*.

11. Algorithm structure.

[15]

- (a) The pseudocode below implements **Dijkstra's algorithm**, but five expressions have been replaced by blanks. Fill in blanks (a)–(e) (shown as dashed lines) with the correct expression or value. [7]

- (a) $dist[s] =$ _____
 (b) condition of the **if**: _____
 (c) $dist[w] =$ _____
 (d) $parent[w] =$ _____
 (e) argument to **add**: _____

```

1  $dist[s] =$  (a)
2  $dist[u] = \infty$  for all  $u \neq s$ 
3  $q = \text{new PriorityQueue}$ 
4  $q.add(s, 0)$ 
5 while  $q$  not empty do
6    $v = q.extractMin()$ 
7   for every edge  $(v, w, c_{v,w})$  do
8     if (b) then
9        $dist[w] =$  (c)
10       $parent[w] =$  (d)
11       $q.add(w, (e))$ 

```

Solution: (a) 0 (b) $dist[w] > dist[v] + c_{v,w}$ (c) $dist[v] + c_{v,w}$ (d) v (e) $dist[w]$

- (b) The seven actions below describe **BFS** (Breadth-First Search), but they are in **scrambled order**. Assign each action a number from 1 to 7 indicating when it is executed (1 = first, 7 = last). [8]

- (A) Set $dist[s] = 0$; $parent[s] = \text{NONE}$; add s to the queue.
 (B) Return the $dist$ and $parent$ arrays.
 (C) Initialize $dist[u] = \infty$ for all vertices u .
 (D) Extract the oldest vertex v from the queue.
 (E) For each neighbor w of v : if $dist[w]$ is not yet set, assign $dist[w] = dist[v] + 1$, $parent[w] = v$, and add w to the queue.
 (F) Repeat steps D and E until the queue is empty.
 (G) Initialize an empty Queue.

Write the correct sequence here: (A) _____ (B) _____ (C) _____ (D) _____ (E) _____ (F) _____ (G) _____

Solution: The correct ordering: **C**(1), **G**(2), **A**(3), **F**(4), **D**(5), **E**(6), **B**(7).

Explanation: first initialize distances to ∞ (C), create the queue (G), initialize source (A), enter the main loop (F), extract oldest vertex (D), process neighbors (E), and finally return results (B).

12. Gradient descent and optimization.

[15]

(a) In supervised machine learning, we minimize a **loss function** $L(\theta)$ over parameters θ .

[4]

(i) Write the **gradient descent update rule**. Define all symbols. (2 pts)(ii) Explain intuitively why we move in the direction of the *negative* gradient. (2 pts)

Solution: (i) $\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$, where θ_t are the current parameters, $\eta > 0$ is the *learning rate*, and $\nabla L(\theta_t)$ is the gradient of the loss at θ_t .

(ii) The gradient $\nabla L(\theta)$ points in the direction of *steepest increase* of L . To *decrease* the loss we move in the opposite direction. Taking a small step in $-\nabla L$ guarantees (for small enough η) that the loss decreases locally.

(b) What happens if the learning rate η is:

[3]

(i) Too small? (1.5 pts)

(ii) Too large? (1.5 pts)

Solution: (i) **Too small:** Training progresses very slowly; many iterations are needed to approach a minimum.

(ii) **Too large:** The update overshoots the minimum; the loss may oscillate or diverge.

(c) Consider the two loss landscapes below.

[4]

(i) Which landscape is **convex**? State one property that distinguishes a convex function. (2 pts)

(ii) For which landscape does gradient descent *always* find the **global** minimum, regardless of starting point? Explain why. (2 pts)

Solution: (i) **Landscape A** is convex. A function is convex if its graph lies at or below the chord connecting any two of its points; equivalently, it has no local minima other than the global minimum.

(ii) Gradient descent always converges to the global minimum on **Landscape A** (convex). On a convex function every local minimum is also the global minimum, so any descent path leads there. On Landscape B (non-convex) gradient descent may stop at the *local* minimum (orange dot) instead of the global one (green dot).

(d) Consider minimizing $L(w) = (w - 5)^2$ using gradient descent with learning rate $\eta = 0.25$, starting at $w_0 = 1$.

[4]

(i) Compute w_1 and w_2 (two gradient steps). (2 pts)(ii) What is the minimum of $L(w)$? Does gradient descent converge to it here? (2 pts)

Solution: $\nabla L(w) = 2(w - 5)$.

$w_1 = 1 - 0.25 \cdot 2(1 - 5) = 1 - 0.25 \cdot (-8) = 1 + 2 = 3$.

$w_2 = 3 - 0.25 \cdot 2(3 - 5) = 3 - 0.25 \cdot (-4) = 3 + 1 = 4$.

The minimum is at $w^* = 5$ with $L(5) = 0$. Yes, GD converges: the distance to w^* halves each step ($4 \rightarrow 2 \rightarrow 1 \rightarrow \dots$), so convergence is geometric but asymptotic.

13. Algorithm design: express lane.

[10]

A motorway has n toll booths numbered 1 to n , each with a waiting time $w[1..n]$ (minutes). A driver must pass through exactly k consecutive booths. Find the starting booth that minimises total waiting time.

The pseudocode skeleton below implements EXPRESSLANE. Five expressions have been replaced by blanks (a)–(e).

- (a) initial value of *windowSum*: _____
 (b) slide the window: _____
 (c) condition to update best: _____
 (d) update *bestSum*: _____
 (e) update *bestStart*: _____

```

1 function ExpressLane( $w[1..n]$ ,  $k$ )
2   windowSum  $\leftarrow$  (a)
3   bestStart  $\leftarrow$  1
4   bestSum  $\leftarrow$  windowSum
5   for  $i \leftarrow 2$  to  $n - k + 1$  do
6     windowSum  $\leftarrow$  (b)
7     if (c) then
8       bestSum  $\leftarrow$  (d)
9       bestStart  $\leftarrow$  (e)
10  return bestStart

```

- (a) Fill in blanks (a)–(e).

[5]

Solution: (a) $\sum_{i=1}^k w[i]$ (b) $\textit{windowSum} - w[i-1] + w[i+k-1]$ (c) $\textit{windowSum} < \textit{bestSum}$
 (d) $\textit{windowSum}$ (e) i

- (b) State the **time** and **space** complexity and briefly justify both.

[2]

Solution: **Time:** $\Theta(n)$ initial sum $\Theta(k)$, then $n - k$ steps of $O(1)$ each. **Space:** $\Theta(1)$.

- (c) Run the completed algorithm on $w = \langle 3, 1, 4, 1, 5, 9, 2, 6 \rangle$ with $k = 3$. Show the window sums and state the best starting index and total time.

[3]

Solution: Window sums: $[3, 1, 4]=8$, $[1, 4, 1]=6$, $[4, 1, 5]=10$, $[1, 5, 9]=15$, $[5, 9, 2]=16$, $[9, 2, 6]=17$.
Best: start = 2, total waiting = 6 min.